

Write control flow with `torch.cond()`, `torch.while_loop()`, or `torch.switch()`; PyTorch captures it as CUDA graph conditional nodes automatically.

Past

CUDA graphs could not capture any data-dependent control flow in PyTorch because that involved synchronizing with the host. Though CUDA graph conditional nodes can run data-dependent control flow fully on the GPU, they were not yet expressible via PyTorch. Users had to split their workload into multiple CUDA graphs, which is intrusive and hard to maintain.

Present

Control flow expressed with `torch.cond()`, `torch.while_loop()`, or `torch.switch()` can now be lowered to a CUDA graph. Outputs must keep a fixed shape across branches and loop iterations. Within that constraint, data-dependent control flow is useful for real workloads, as the examples here demonstrate.



Further examples

cond optimizer skip
switch embedding buckets
switch top-1 MoE
ibm.biz/controlflow-in-cuda-graphs

Support matrix

Control-flow ops and the CUDA graph node each lowers to

	cond IF / ELSE	while_loop WHILE	switch SWITCH
PyTorch 2.14	✓	✓	✓
CUDA graph support	✓	✓	🚧
In-place input mutation*	✓	✓	✓
Minimum CUDA	12.4	12.4	12.8
Inductor backend	✗	✗	✗

✓ supported in PyTorch 2.14 🚧 PR open, not yet merged

* Inputs mutated in place under `torch.no_grad()` or `torch.inference_mode()`.
`switch` is `torch._higher_order_ops.switch`; CUDA graph capture needs #189461.
Inductor runs the control flow on the host; it emits no conditional nodes yet.
Checked on PyTorch main (91a0eda); every feature above is in 2.14.

01 Skip optimizer if gradient is non-finite

`torch.cond()` → IF/ELSE conditional node

```
def safe_sgd(param, grad, lr):  
    finite = torch.isfinite(grad).all()  
  
    def update():  
        param.sub_(lr * grad)
```

```
torch.cond(finite, update, lambda: None, ())
```

Capture once, then replay:

```
step = torch.compile(safe_sgd, backend="cudagraphs")  
step(param, grad, lr) # replays one graph
```

The predicate never leaves the GPU: no `.item()`, no CPU sync.

The same graph updates `param` or leaves it unchanged on each replay.

Before: a Python `if` on this flag forces a CPU sync and breaks capture.

02 Embedding bags without maximum padding

`torch.switch()` → SWITCH conditional node

Recommender system requests vary widely in token count, but each embedding bag returns a fixed-width vector. Select a token-count bucket on the GPU and pad only to that bucket's capacity.

```
from torch._higher_order_ops import switch
```

```
def embedding_bag(table, ids, n):  
    caps = (32, 128, 512, 4096)  
    bucket = sum((n > c).to(torch.int32)  
                 for c in caps[:-1])  
  
    def reduce(cap):  
        pos = torch.arange(cap, device=ids.device)  
        rows = table[ids[:cap]]  
        return (rows * (pos < n)[:, None]).sum(0)
```

```
branches = [lambda c=c: reduce(c) for c in caps]  
return switch(bucket, branches, ())
```

SWITCH: token-count bucket

`n = 80`

32 tokens

128 tokens

512 tokens

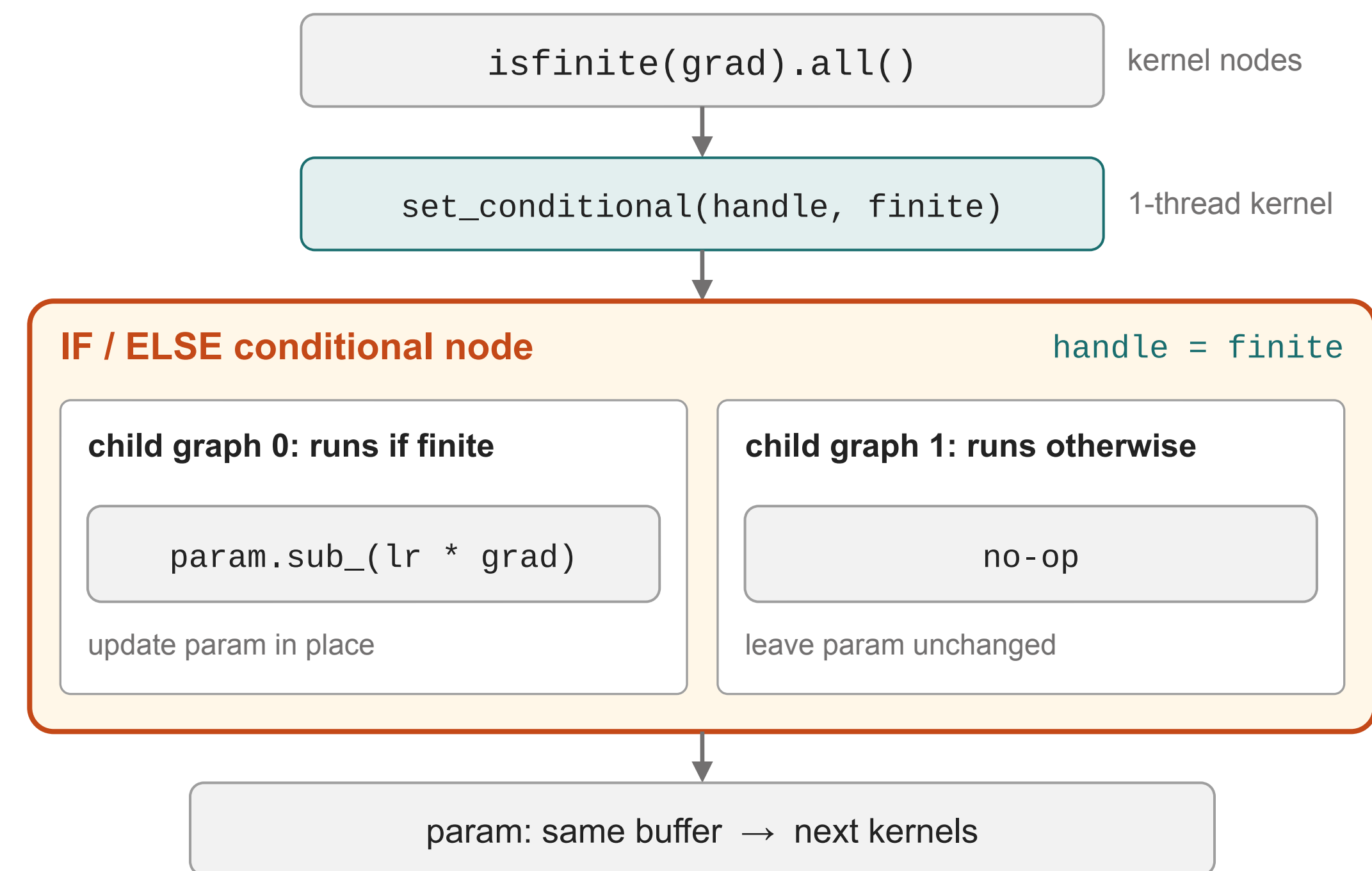
4096 tokens

Every branch returns [D]; 80 tokens use the 128-token branch.

ids: fixed 4,096-slot buffer; $0 \leq n \leq 4,096$; output: [D].

The captured CUDA graph

What `safe_sgd` becomes: one graph, GPU-side branch



03 Lossless MoE routing with software backpressure

`ep8_moe: torch.while_loop() + grouped_mm()`

EP=8; 4,096 tokens/rank; 512 slots/source; zero tokens dropped.

1. All ranks agree on the number of rounds

```
total_rounds = sync_all_reduce_max(local_rounds)
def cond(iteration, done, output):
    return iteration < total_rounds
```

2. Group packed tokens by expert; run grouped GEMM

```
counts = (  
    (packed_experts[:, None]  
     == local_expert_ids[None, :])  
    & packed_valid[:, None]  
) .sum(0)  
offsets = torch.cumsum(counts, 0, dtype=torch.int32)  
packed_output = torch.nn.functional.grouped_mm(  
    packed_tokens,  
    local_weights.transpose(-2, -1),  
    offs=offsets,  
)
```

3. Repeat bounded all-to-all rounds on the GPU

```
iteration, _, output = torch.while_loop(  
    cond, body, (iteration, done, output))  
return output, iteration
```

Excerpts: routing, packing, returns, and capture helpers omitted.